

 Powered by Arizona State University®	Universidad Internacional del Ecuador	
	Escuela de Ciencias de la Computación Carrera Ingeniería en Sistemas de Información	
Nombre: ZACK DERECK RODRIGUEZ ESPINOZA		Trabajo: Autónomo / Práctico
Materia: PROGRAMACION ORIENTADA A OBJETOS		Semestre: Tercero
Fecha: 30/6/2026		Profesor: DAVID HERNAN GUEVARA GUEVARA

1. Objetivo

El propósito de esta práctica es comprobar de forma práctica cómo funciona la encapsulación en Go al utilizar el paquete **banco** desde dos escenarios diferentes. El primero corresponde al mismo paquete (**banco**), mediante un archivo de pruebas internas, y el segundo a un paquete externo (**banco_test**). Con esto se busca verificar cuáles funciones, métodos y campos pueden utilizarse desde fuera del paquete y cuáles permanecen ocultos, de acuerdo con la regla de Go que diferencia los identificadores públicos y privados según si comienzan con mayúscula o minúscula.

2. Estructura del proyecto

El proyecto está organizado en dos carpetas de pruebas que trabajan sobre el mismo paquete de producción llamado **banco**:

CUENTACUENTATALLER19

banco

 cuenta.go // código de producción

 cuenta_test.go // pruebas internas (package banco)

banco_test

 cuenta_externo_test.go // pruebas externas (package banco_test)

go.mod

Dentro del archivo **cuenta.go** se encuentra la estructura **Cuenta**, la cual posee un campo público llamado **Nombre** y un campo privado denominado **saldo**. Además, incluye funciones y métodos públicos como **NuevaCuenta**, **Depositar** y **ObtenerSaldo**, junto con otros

REINVENTEMOS
EL FUTURO

elementos privados como **calcularInteres** y **nuevaCuentaPrivada**, que solo pueden utilizarse dentro del mismo paquete.

3. Cambios realizados en `cuenta_externo_test.go`

Para desarrollar la actividad se completó el archivo **banco_test/cuenta_externo_test.go**, ya que inicialmente únicamente contenía la declaración del paquete. Se agregaron dos funciones de prueba con el objetivo de comprobar el funcionamiento de la API pública del paquete **banco** cuando es utilizada desde un paquete externo.

- **TestAccesoPublico:** contiene cuatro subpruebas mediante `t.Run`. En ellas se verifica el uso correcto de los elementos públicos, como **NuevaCuenta**, **Nombre**, **Depositar** y **ObtenerSaldo**. También se incluyen comentarios y un `t.Log` para indicar cuáles accesos no serían válidos desde un paquete externo, como **nuevaCuentaPrivada**, el campo **saldo** y el método **calcularInteres**.
- **TestFlujocompletoPublico:** realiza una prueba completa utilizando únicamente la interfaz pública del paquete. En esta prueba se crea una cuenta, se realizan dos depósitos y posteriormente se comprueba que tanto el saldo como el nombre sean los esperados.

A diferencia de **cuenta_test.go**, que pertenece al mismo paquete **banco**, en este archivo no es posible acceder directamente a miembros privados. Si se intentara hacerlo, el código no llegaría a compilar, por lo que dichos accesos únicamente se muestran como comentarios explicativos.

4. Ejecución de las pruebas

4.1 Comando ejecutado

No cambies este comando.

```
go test ./... -v -cover
```

REINVENTEMOS
EL FUTURO

4.2 Salida completa de la ejecución

Aquí deja exactamente la misma salida que obtuviste al ejecutar el comando. No cambies nada de esta sección, ya que corresponde a la evidencia de la práctica.

5. Comparación entre la ejecución anterior y la nueva

La ejecución anterior corresponde a las pruebas que ya estaban implementadas en **banco/cuenta_test.go**, las cuales se ejecutan dentro del mismo paquete que el código de producción. En cambio, la nueva ejecución pertenece al archivo **banco_test/cuenta_externo_test.go**, donde las pruebas se realizan desde un paquete externo.

La tabla se mantiene exactamente igual. No la modifiques.

6. Análisis

6.1 ¿Qué nuevas pruebas aparecen en la ejecución?

En esta práctica se incorporaron dos nuevas funciones de prueba: **TestAccesoPublico** y **TestFlujoCompletoPublico**. La primera está formada por cuatro subpruebas creadas con `t.Run`, mientras que la segunda comprueba el funcionamiento completo de la API pública. Ambas se encuentran en el archivo **banco_test/cuenta_externo_test.go**, el cual originalmente solo contenía la declaración del paquete y fue completado durante el desarrollo de la actividad.

6.2 ¿Qué métodos o funcionalidades adicionales se están probando?

REINVENTEMOS
EL FUTURO

Las nuevas pruebas no agregan funcionalidades diferentes al programa, sino que verifican el comportamiento de la interfaz pública del paquete desde la perspectiva de un usuario externo. Para ello se utilizan elementos como **NuevaCuenta**, **Depositar**, **ObtenerSaldo** y el campo **Nombre**, accediendo a ellos mediante la importación del paquete **CuentaCuenta/banco**. De esta manera se demuestra que la API pública funciona correctamente incluso cuando es utilizada desde otro paquete.

6.3 ¿Cómo se refleja en los resultados el hecho de que los nuevos métodos son privados?

Esto puede observarse de dos formas.

La primera ocurre durante la compilación. Si se intentara acceder a elementos privados como **banco.nuevaCuentaPrivada**, **cuenta.saldo** o **cuenta.calcularInteres**, el proyecto no compilaría. Por esta razón esas líneas permanecen comentadas dentro del archivo de pruebas y sirven únicamente como referencia de lo que no está permitido.

La segunda se aprecia en la salida de **go test**. En la subprueba **NO_puedo_usar_función_privada_nuevaCuentaPrivada** no se ejecuta realmente la función privada; únicamente se muestra un mensaje mediante `t.Log` indicando que esa línea produciría un error de compilación si se descomentara. El resultado **PASS** demuestra que el comportamiento esperado es precisamente impedir ese acceso.

En cambio, las pruebas internas del paquete **banco** sí pueden utilizar directamente **nuevaCuentaPrivada**, modificar el campo **saldo** y llamar al método **calcularInteres**, ya que pertenecen al mismo paquete donde fueron definidos.

6.4 ¿Qué diferencias observas en la cobertura de pruebas entre ambas ejecuciones?

La diferencia principal se encuentra en el reporte de cobertura.

En el paquete **banco** se obtiene una cobertura del **100 %**, porque las pruebas se ejecutan dentro del mismo paquete que contiene el código de producción. Esto permite que la

REINVENTEMOS
EL FUTURO

herramienta de cobertura registre todas las sentencias ejecutadas, incluyendo las correspondientes a métodos privados.

Por otro lado, el paquete **banco_test** muestra el resultado **coverage: [no statements]**. Esto sucede porque la cobertura se calcula por paquete y **banco_test** únicamente contiene archivos de prueba, sin código de producción propio. Aunque estas pruebas ejecutan funciones del paquete **banco**, dichas sentencias no se contabilizan dentro del paquete externo.

Si se desea medir cuánto código del paquete **banco** es cubierto por las pruebas externas, sería necesario utilizar el siguiente comando:

```
go test -coverpkg=CuentaCuenta/banco ./banco_test/...
```

6.5 ¿Qué aprendizaje obtienes sobre la encapsulación en Go a partir de este ejercicio?

Este ejercicio permitió entender que Go maneja la encapsulación de una forma sencilla, pero estricta. La visibilidad de funciones, métodos, campos y tipos depende únicamente de si el nombre inicia con una letra mayúscula o minúscula. Los identificadores que comienzan con mayúscula son públicos y pueden utilizarse desde otros paquetes, mientras que los que empiezan con minúscula permanecen restringidos al paquete donde fueron creados.

También se pudo comprobar que dos archivos pueden estar en la misma carpeta y aun así pertenecer a paquetes diferentes, ya que lo que realmente determina el acceso es la declaración `package` y no la ubicación física del archivo.

Las pruebas internas permiten acceder a todos los elementos del paquete, incluidos los privados, lo que resulta útil para verificar detalles de implementación. En cambio, las pruebas externas funcionan como lo haría cualquier aplicación que importe el paquete, ya que únicamente tienen acceso a la API pública.

Finalmente, quedó claro que Go detecta cualquier intento de acceder a miembros privados durante la compilación. Esto garantiza una encapsulación sólida y obliga a que los usuarios del paquete interactúen únicamente con los elementos públicos que fueron diseñados para ese propósito.

REINVENTEMOS
EL FUTURO

7. Conclusiones

La práctica permitió comprobar de forma clara cómo funciona la encapsulación en Go al realizar pruebas desde un paquete interno y desde uno externo. Aunque ambos casos verifican las mismas operaciones principales, como crear una cuenta, realizar depósitos y consultar el saldo, el nivel de acceso es diferente según el paquete desde el que se ejecutan las pruebas.

Además, se confirmó que la API pública del paquete está correctamente definida mediante elementos como **NuevaCuenta**, **Depositar**, **ObtenerSaldo** y **Nombre**, mientras que los detalles internos, como **saldo**, **calcularInteres** y **nuevaCuentaPrivada**, permanecen protegidos y solo pueden utilizarse dentro del propio paquete. Esto demuestra la importancia de la encapsulación para mantener un código más seguro, organizado y fácil de mantener.

8. Entregables

Esta sección prácticamente no necesita cambios, solo una ligera modificación en la redacción:

- Código fuente del proyecto **poo_taller17**, incluyendo el archivo **banco_test/cuenta_externo_test.go** completamente desarrollado.
 - Captura o salida de la ejecución del comando **go test ./... -v -cover**, presentada en la sección 4.2 del informe.
 - Informe final con el análisis comparativo solicitado en la actividad.
-

REINVENTEMOS
EL FUTURO

REINVENTEMOS
EL FUTURO

